# SIMATS ENGINEERING

## CSA10 – SOFTWARE ENGINEERING

## ASSIGNMENT 1

# DESIGN, SIMULATE AND VALIDATE AN AGILE DELIVERY STRATEGY

## FOR THE OMNICHANNEL ORDER FULFILMENT PLATFORM

| **Student Details** | |
|---|---|
| Name | Karnam Harshith |
| Register Number | 192511384 |
| Programme | B.Tech – Computer Science and Engineering |
| Course | CSA10 – Software Engineering |
| Assignment | Assignment 1 |

*Individual Assignment Report*

# Contents and Report Approach

| Section | Coverage |
|---|---|
| Task 1 – Problem investigation | Stakeholder needs, questions, simulated evidence, scope, feasibility and success criteria |
| Task 2 – Agile and Lean approach | Scrum versus Kanban/Lean, selection and operating model |
| Task 3 – Product backlog | Epics, stories, acceptance criteria, priorities, dependencies, DoR and DoD |
| Task 4 – Release plan | Capacity assumptions, four sprint goals, workflow and WIP controls |
| Task 5 – Sprint 1 simulation | Bottleneck prediction, event log, indicators and improvement |
| Task 6 – Changed-condition redesign | Impact, objectives, scaled coordination, revised backlog and second simulation |
| Task 7 – Technical reflection | Decision, evidence, vulnerability, ethics and next investigation |
| Appendices | Jira configuration proposal, traceability, assumptions and references |

## How the report is prepared

This report follows the seven tasks and evidence expectations in the assignment brief. The retailer's scenario is treated as the fixed case. Since access to actual retailer employees and a live Jira project was not provided for this individual report, stakeholder findings and delivery events are explicitly labelled as simulated planning evidence. They are not presented as interviews, production measurements or completed team work.

The proposed estimates, capacities, thresholds and sprint results are planning assumptions created to make the strategy testable. They should be replaced with real observations if the project is carried out with an actual development team.

## Planning principle

The first release is not intended to reproduce every proposed module at full scale. It is designed around one complete and auditable order journey: checkout, stock reservation, payment status, picking, dispatch, tracking, support lookup and a controlled return/refund path. Features that do not protect this journey are kept outside the initial release.

# Task 1 – Investigate the Problem and Define Delivery Outcomes

## 1.1 Understanding the scenario

The retailer currently sells through a website, a mobile application and twelve physical stores. These channels create a shared fulfilment problem because a customer may place an order in one channel while stock is handled by a warehouse or store. If the systems do not share reliable order and stock information, the retailer can accept orders that cannot be fulfilled, delay refunds or make it difficult for support staff to explain what happened.

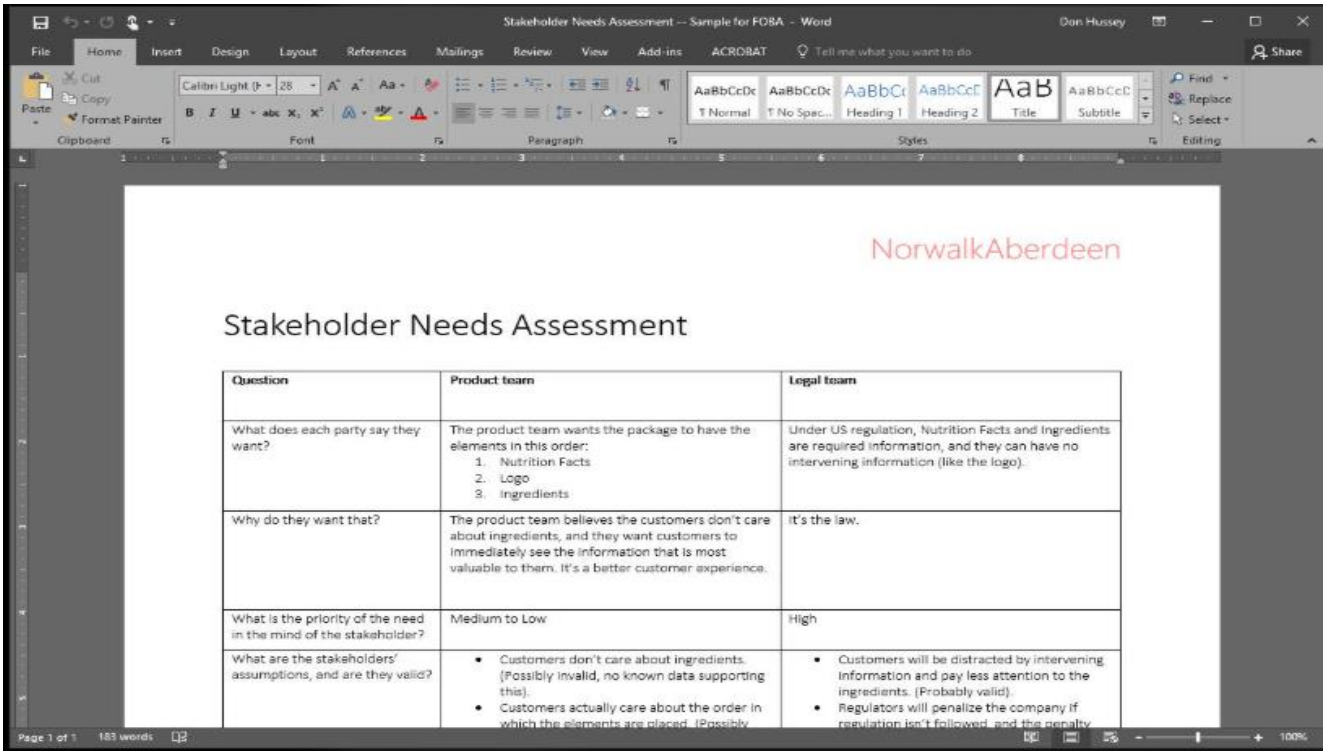
The assignment identifies three important operational viewpoints. Warehouse pickers need accurate reservation information and practical pick lists. Support agents need a clear view of the order, delivery and return status. Finance reviewers need evidence that refunds are authorized and that payment records can be reconciled. The platform must connect these needs rather than deliver isolated screens.

## 1.2 Problem statement

The retailer needs a controlled omnichannel order fulfilment process that maintains a consistent record from checkout to delivery and, where required, to an approved refund. The first release must reduce avoidable failed orders and unclear ownership without depending on unavailable external integrations. It must also provide enough audit information for finance and enough status visibility for support.

## 1.3 Questions used for investigation

1. What are the most common reasons an order is accepted but later cannot be fulfilled?
2. At what point do warehouse pickers currently receive confirmed stock and pick instructions?
3. What information does a support agent need to answer an order or return enquiry without contacting several teams?
4. Who is allowed to approve a refund, and what evidence must be retained for reconciliation?
5. What should the platform do when the payment gateway is unavailable or returns an uncertain response?
6. Which order states must be visible to customers, store staff and internal users?

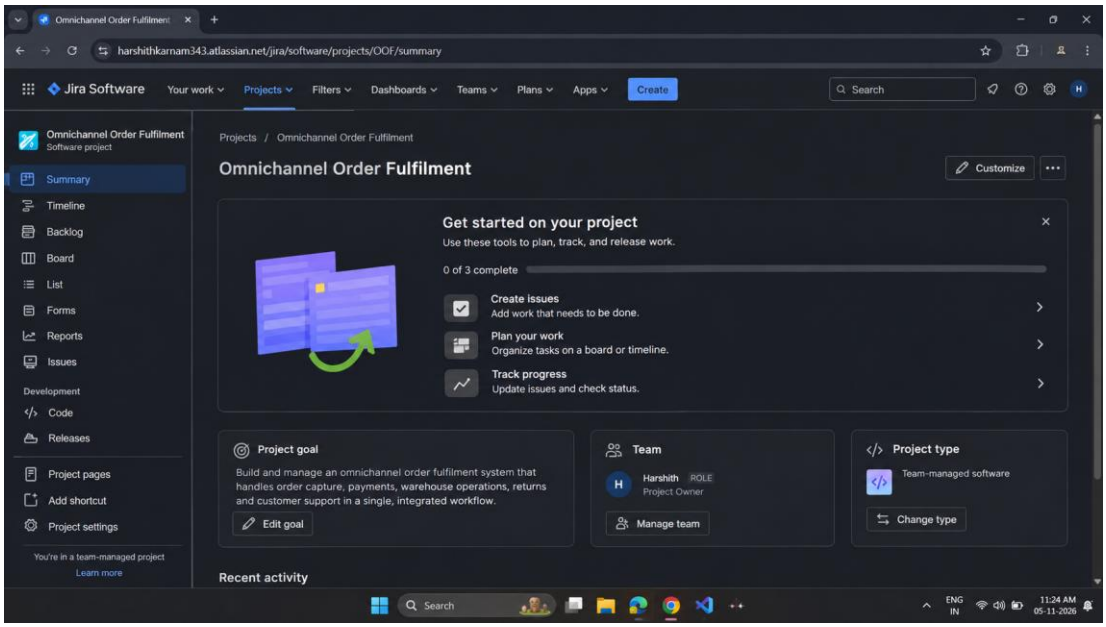


# Task 1 – Stakeholder Findings

## 1.4 Initial evidence (simulated)

For this report, a small simulated stakeholder review was used because direct access to the retailer's staff was not available. The responses below are working assumptions based on the scenario, not actual interview quotations.

| Stakeholder | Simulated finding | Requirement derived |
|---|---|---|
| Customer | Wants confirmation that the order is accepted and a dependable delivery/return status. | Order confirmation, tracking timeline and clear exception message. |
| Warehouse picker | Needs a pick list that reflects reserved quantities and does not show cancelled orders as active work. | Reservation-aware pick list and controlled picking status. |
| Support agent | Cannot confidently answer when order, dispatch and return information are spread across screens. | Single order lookup showing current status and history. |
| Finance reviewer | Needs to distinguish a requested refund from an approved and processed refund. | Role-based approval, refund audit trail and payment reference. |
| Store manager | Needs visibility of store-related stock changes and responsibility for corrections. | Stock adjustment request with reason and responsible user. |
| Development team | Cannot complete a real gateway integration before sandbox access starts in week four. | Payment adapter and safe simulated payment mode before integration. |




## 1.5 Main findings

- The most important risk is an incomplete or inconsistent order record, not the absence of advanced analytics.
- Stock reservation must happen before a pick task is treated as valid work.
- Payment retries must be idempotent so a timeout does not create a second charge.
- Refund approval should remain a human decision, even if the system automates validation and record keeping.
- Support visibility depends on reliable status events from earlier stages of the order journey.

The findings indicate that the first release should be organized around the transaction flow and its controls. A storefront-only release would not solve the operational problem, while a full implementation of all nine modules would create too much integration work for the available time.

# Task 1 – Product Goal, Scope and Feasibility

## 1.6 Product goal

Deliver, within eight weeks, a usable first version of an omnichannel fulfilment platform that records an order consistently from checkout through reservation, payment status, picking, dispatch and tracking, and supports an auditable, authorized return/refund process.

## 1.7 First-release scope

| Included in first release | Deferred or excluded |
|---|---|
| Synthetic product catalogue and cart/checkout flow. | Full production storefront redesign and personalization. |
| Local order record and stock reservation against simulated inventory. | Live synchronization with all twelve stores and every warehouse system. |
| Payment adapter with sandbox integration when available; safe simulated mode otherwise. | Production card storage or handling of real card data. |
| Picker dashboard with reservation-linked pick list and dispatch status. | Advanced barcode automation before scanner device arrives and is tested. |
| Customer/support order timeline and return request. | Omnichannel marketing, loyalty and recommendation features. |
| Refund request, role-based human approval and audit history. | Automatic refund approval and complex finance automation. |
| Basic operational report for order and refund status. | Predictive analytics and machine-learning demand forecasting. |

## 1.8 Measurable success criteria

| Measure | Planning target for release validation | How it will be checked |
|---|---|---|
| Order journey completion | At least 9 of 10 scripted test orders reach the correct final state. | Repeatable end-to-end test cases. |
| Stock control | No test order reserves more than available simulated stock. | Boundary and concurrent reservation tests. |
| Payment retry safety | Repeated request with the same idempotency key creates no duplicate payment record. | Timeout and retry test. |
| Refund audit | Every approved test refund records requester, approver, amount, reason and timestamp. | Audit record inspection. |
| Support visibility | A support user can retrieve order, dispatch and return status from one lookup. | Role-based usability walkthrough. |

These are acceptance targets for the prototype release, not claims about production service-level performance.

# Task 1 – Feasibility Assessment

## 1.9 Technical feasibility

A small web application with a modular backend and relational database is technically feasible for a seven-member team over four two-week sprints, provided that the first release uses synthetic data and controlled integrations. The platform can represent orders, order items, stock reservations, payment attempts, shipments, returns and refund approvals as related records. A payment adapter can isolate the gateway-specific code so that the rest of the system can be tested before the sandbox becomes available.

The main technical uncertainty is payment behaviour when the gateway sandbox is delayed or responds slowly. The proposed design stores the local order before attempting payment, assigns a unique idempotency key to each payment operation and records the result of each attempt. A retry with the same key should return the existing result rather than create a second charge. The scanner device arrives in week five, so picking can initially be tested through manual item confirmation and then extended to scanner input.

## 1.10 Operational feasibility

The seven-person cross-functional team can build a narrow workflow if responsibilities are clear and if warehouse and finance stakeholders use their limited ninety minutes per week for prepared clarification and review sessions. The product owner or nominated business representative must make priority decisions between meetings. The team should avoid waiting for a large group to approve every small design detail.

## 1.11 Financial and schedule feasibility

The assignment does not provide a project budget, staff salary rates or vendor quotations. Therefore, a precise financial return calculation would not be reliable. The strategy limits cost exposure by selecting one project-management tool, using synthetic test data, avoiding paid add-ons where possible and deferring low-value modules. The fixed eight-week deadline is feasible only if the scope is protected and the team does not attempt a full multi-region production rollout.

## 1.12 Key assumptions and risks

- One product owner can make day-to-day priority decisions and obtain stakeholder answers.
- Team availability is assumed to average 25 focused delivery hours per person per week after meetings and support work.
- Gateway sandbox access begins in week four; scanner hardware is available in week five, as stated in the scenario.
- All data used in the prototype is synthetic; no real customer or payment-card information is used.
- External store and warehouse integrations are represented by stable interfaces or test adapters in the first release.

# Task 2 – Design and Justify the Project Management Approach

## 2.1 Approach A: Scrum with Lean practices

Under Scrum, the team works in two-week sprints with a product backlog, sprint goal, sprint planning, daily coordination, sprint review and retrospective. The product owner orders the backlog and the development team selects a realistic set of stories. The sprint goal gives the team a shared outcome rather than a collection of unrelated tasks.

Lean practices are added to reduce waiting and rework. Work is kept small, work in progress is limited, blocked items are made visible and the team aims to finish started work before starting more. Acceptance criteria and early testing reduce the chance that a story is considered complete only at the end of the sprint.

## 2.2 Approach B: Kanban/Lean flow

A Kanban approach would maintain a continuously ordered work queue and move stories through defined workflow states. The team would pull new work only when capacity is available. Cycle time, throughput, blocked time and work in progress would be reviewed regularly. This approach can be useful when support work and urgent changes arrive unpredictably.

For this scenario, a pure continuous-flow model has a disadvantage: the retailer has a fixed eight-week release date and needs coordinated review points for warehouse and finance stakeholders. Without sprint goals, it may be harder to reserve a complete end-to-end outcome for each review.

## 2.3 Comparison

| Factor | Scrum with Lean | Kanban/Lean |
|---|---|---|
| Planning | Two-week goals and sprint commitments based on capacity. | Continuous replenishment and pull-based selection. |
| Changing requirements | Changes are ordered in the backlog; sprint goal is protected where possible. | New work can enter when a slot is available, subject to policy. |
| Cross-functional review | Scheduled sprint reviews provide predictable stakeholder checkpoints. | Reviews can be frequent, but stakeholder availability must be arranged. |
| Flow control | WIP limits and blocked-work policies are applied within the sprint. | WIP limits are central to the whole workflow. |
| Main risk | Sprint scope may be disrupted by urgent requests. | A fixed release outcome may become less visible without explicit milestones. |
| Fit for this case | Strong fit for four planned two-week release increments. | Useful supporting method for controlling daily flow. |

| METHODOLOGY | STRENGTHS | WEAKNESSES | KEY DIFFERENTIATOR |
|---|---|---|---|
| Agile | Delivers products fast | Requires constant refinement | Strategic Alignment |
| Lean | Prioritizes value creation | Functional capabilities are not centralized | Delivering Value |
| Waterfall | Easy to control | Lacks innovation | Risk Management |
| Kanban | Easy to compartmentalize | Not intuitive | Increased Visibility |

# Task 2 – Selected Operating Model

## 2.4 Selected approach

I would use Scrum with Lean flow controls. The four two-week sprints match the eight-week release window and provide regular points to demonstrate a working order journey. Kanban ideas are used inside each sprint to control work in progress, expose blockers and prevent unfinished tasks from accumulating.

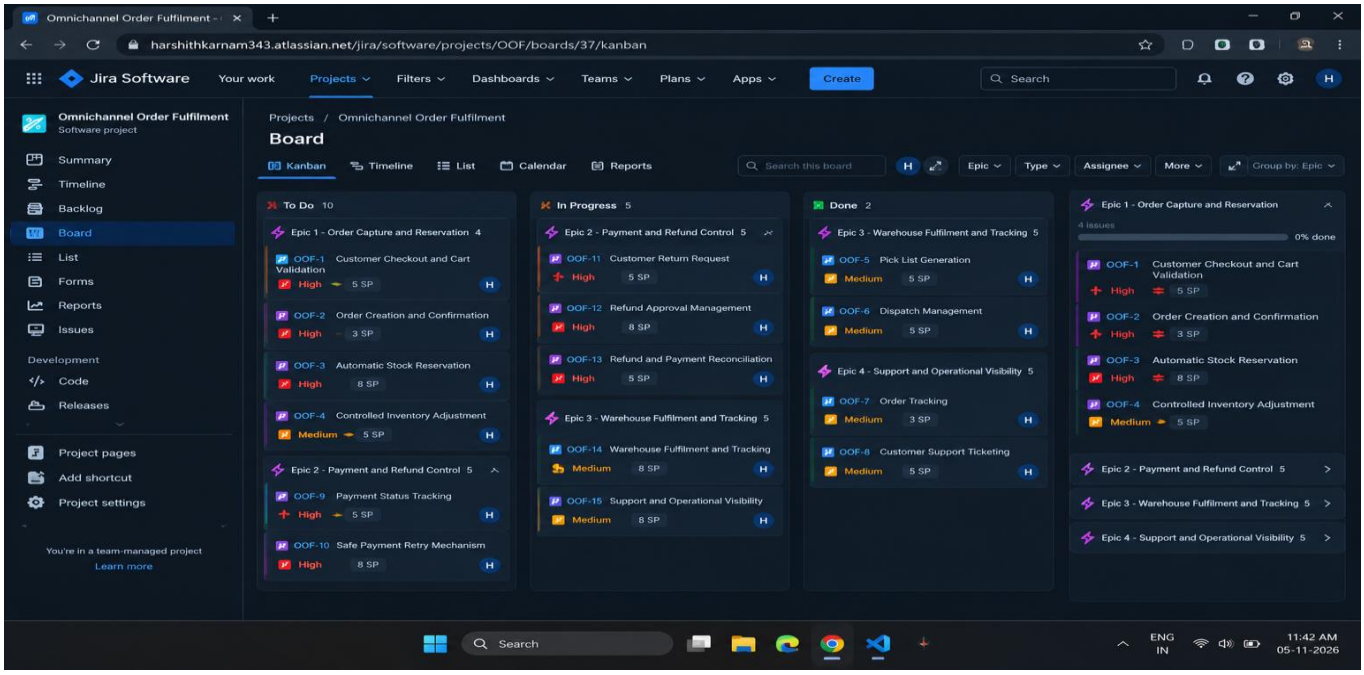
## 2.5 Seven-member team responsibilities

| Role | Main responsibility | Decision or output |
|---|---|---|
| Product owner / business analyst | Clarify user needs, order backlog and confirm business acceptance. | Prioritized backlog and sprint goal. |
| Scrum facilitator / delivery coordinator | Facilitate ceremonies, remove process blockers and maintain delivery visibility. | Impediment log and team coordination. |
| Backend engineer | Order, stock, payment and refund services. | APIs, transaction rules and data persistence. |
| Frontend engineer | Customer checkout, support console and picker screens. | Accessible user interface and status views. |
| Integration engineer | Payment adapter, event handling and scanner integration. | Gateway interface and integration tests. |
| QA / test engineer | Test design, automation and defect verification. | Test evidence and regression results. |
| Database / DevOps engineer | Data model, environments, deployment pipeline and audit persistence. | Schema, build pipeline and deployment checks. |

These are proposed responsibilities for planning. In a real small team, people may share skills, but every story must still have one accountable owner and named reviewers.

## 2.6 Ceremonies and stakeholder participation

- Sprint planning: first morning of each sprint; select work against available capacity and dependencies.
- Daily 15-minute coordination: focus on progress toward the sprint goal, blockers and hand-offs.
- Weekly 90-minute stakeholder window: prepare a short demonstration and a decision list for warehouse and finance representatives.
- Sprint review: demonstrate working stories using synthetic orders and capture acceptance or change requests.
- Retrospective: review flow indicators and agree one improvement that can be checked in the next sprint.



# Task 2 – Lean Controls and Tool Selection

## 2.7 Lean practices

| Lean practice | How it is applied | Expected effect |
|---|---|---|
| Visualize work | Use Jira columns and labels for each workflow state and blocked reason. | Makes waiting and ownership visible. |
| Limit WIP | Maximum 2 stories in development and 2 in testing for the whole team. | Reduces multitasking and unfinished work. |
| Manage flow | Review aging work and blocked time in daily coordination. | Encourages finishing and unblocking. |
| Reduce hand-offs | Backend, frontend and QA agree API contracts before implementation. | Avoids repeated clarification between roles. |
| Build quality in | Acceptance criteria, peer review, automated tests and DoD checks. | Finds defects before sprint review. |
| Continuous improvement | Select one retrospective action and check its effect next sprint. | Prevents process changes from becoming vague promises. |

## 2.8 Project-management tools considered

| Tool | Strengths for this assignment | Limitations / response |
|---|---|---|
| Jira | Epics, stories, priorities, estimates, dependencies, configurable workflow, boards and reports. Supports Scrum and Kanban views. | Some advanced reporting and administration can be complex; use a simple project configuration and avoid unnecessary add-ons. |
| Trello | Easy visual cards, lists, labels and simple collaboration; low learning effort. | Complex story relationships, release planning and detailed dependency reporting are less convenient; use checklists or external documents if selected. |

## 2.9 Selected tool: Jira

Jira is selected because the assignment specifically requires a structured backlog with epics, user stories, acceptance criteria, priorities, estimates and dependencies. A Scrum board can show sprint progress while issue links and labels make integration risks visible. Jira reports can help compare planned and completed work, although the team should interpret them alongside the event log rather than treating a chart as proof of quality.

The proposed configuration is intentionally small: one project, four epics, a manageable set of stories, one Scrum board, a few workflow statuses, a blocked label and sprint-level reports. No claim is made that a live Jira project has been created in this report.

# Task 3 – Construct a Prioritized Product Backlog

## 3.1 Backlog structure

The backlog is divided into four epics that map to the operational journey and stakeholder needs. The priority order uses MoSCoW at the release level and a simple value-risk-dependency discussion within each group. Must-have items are those required to complete or control the order journey. Should-have items improve usability or resilience but can be reduced if capacity is lower than expected.

| Epic ID | Epic | Purpose | Release decision |
|---|---|---|---|
| EP-01 | Order capture and reservation | Create a valid order and reserve available stock. | Retain – Must |
| EP-02 | Payment and refund control | Record payment attempts and controlled refund approvals. | Retain – Must |
| EP-03 | Warehouse fulfilment and tracking | Provide pick work, dispatch update and status timeline. | Retain – Must |
| EP-04 | Support and operational visibility | Allow support lookup and basic operational reporting. | Retain limited scope – Must/Should |

## 3.2 Deferred modules

A full analytics suite, machine-learning demand forecast, loyalty functions, advanced catalogue personalization and live integration with all twelve stores are deferred. They are not ignored; they are excluded because they do not directly complete the first order-to-refund journey and would introduce extra data, integration and validation work. Barcode scanning is a planned enhancement after the test device arrives in week five, but manual pick confirmation remains the fallback.

## 3.3 Prioritization method

MoSCoW is used because the release deadline is fixed and the scope needs a clear boundary. The product owner checks each proposed story against three questions: does it protect a required step in the order journey, does another story depend on it, and is it testable within the available capacity? Security, privacy, authorization and audit controls are not treated as optional polish.

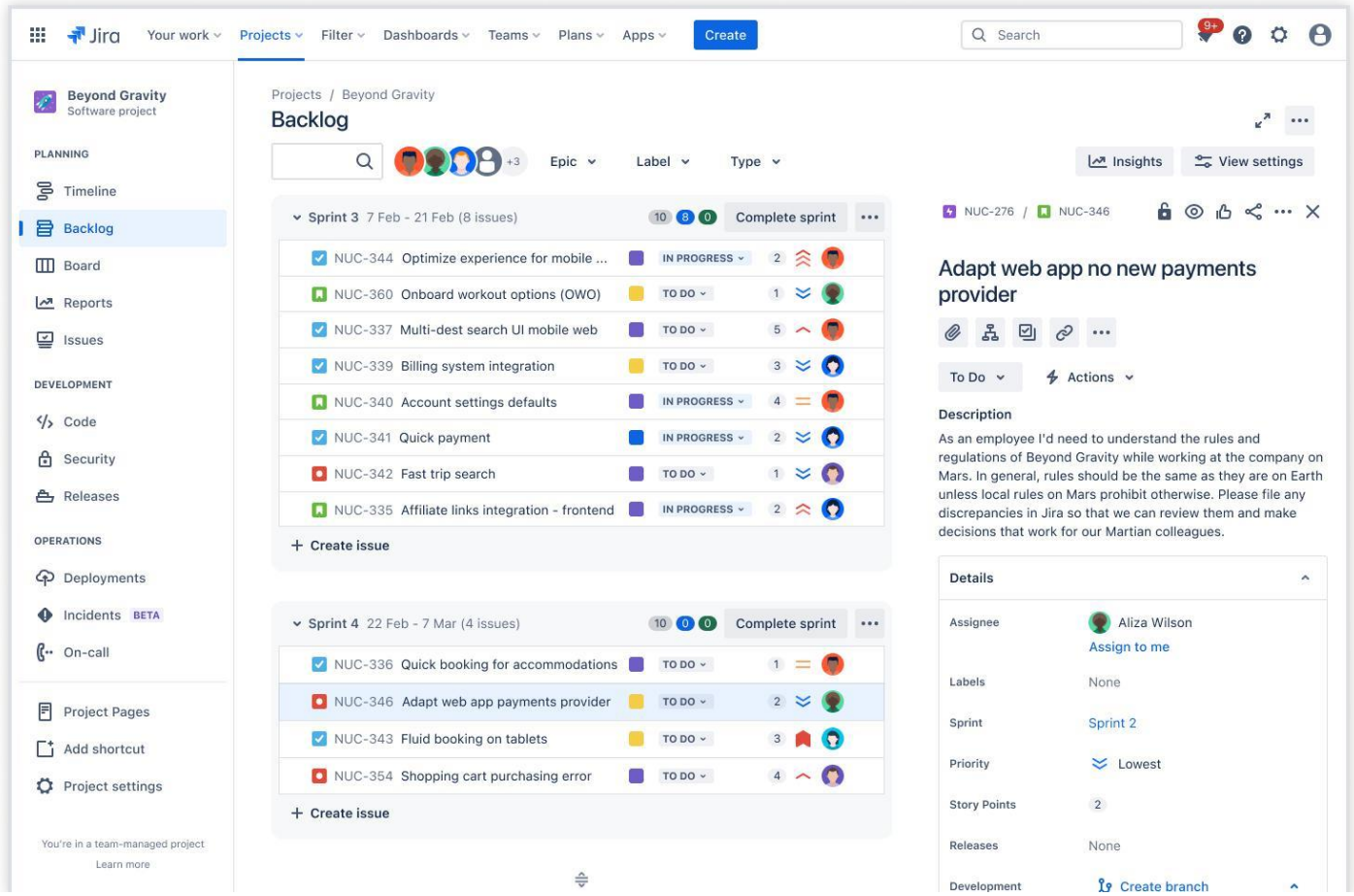
# Task 3 – Backlog Stories: Order and Inventory

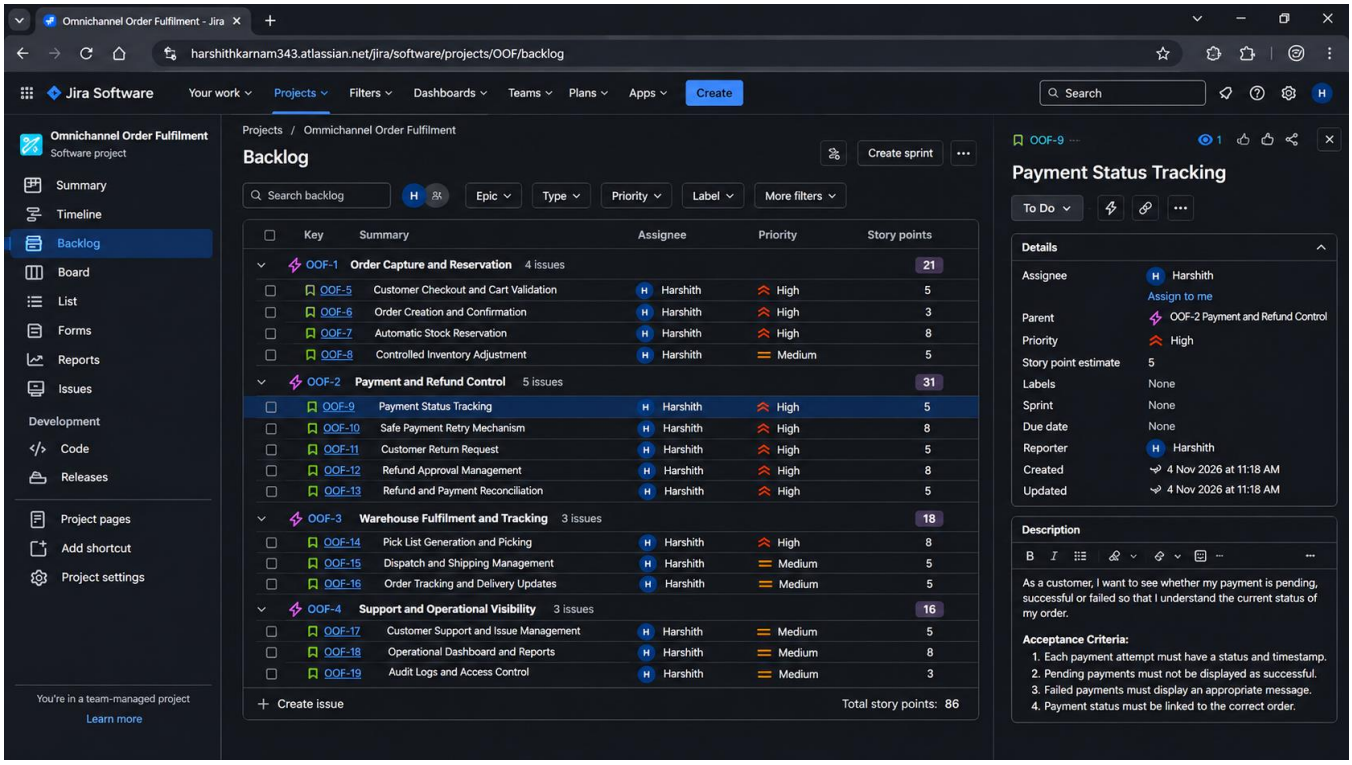
## 3.4 Epic EP-01: Order capture and reservation

| Story | User story and acceptance criteria | Priority / estimate / dependency / owner |
|---|---|---|
| US-01 | As a customer, I want to review my cart and submit checkout so that I can create an order. AC: required item and quantity are validated; displayed total is recalculated by the server; invalid or unavailable items are rejected with a clear message. | Must / 5 points / catalogue data / Frontend + Backend |
| US-02 | As a customer, I want an order reference after checkout so that I can identify my purchase. AC: unique order ID is created; order and items persist; confirmation is shown only after the local order record is saved. | Must / 3 points / US-01 / Backend |
| US-03 | As a system, I want to reserve stock before confirming fulfilment so that two orders do not consume the same available quantity. AC: reservation cannot exceed available stock; repeated request is safe; failed reservation leaves a clear order state. | Must / 8 points / US-01, US-02 / Backend + QA |
| US-04 | As a store manager, I want stock corrections to require a reason and user identity so that changes can be traced. AC: correction records old and new quantity, reason, user and timestamp; ordinary users cannot directly overwrite stock. | Should / 5 points / stock model / Backend + QA |

## 3.5 Inventory rules

Available stock is calculated as recorded on-hand quantity less active reservations. A reservation is created in a database transaction so that two concurrent checkouts cannot both reserve the same final unit. A reservation expires or is released according to a defined order-cancellation rule. For the prototype, the expiry period is assumed to be fifteen minutes after an unpaid checkout; this is a test rule, not a retailer policy.






# Task 3 – Backlog Stories: Payments and Refunds

## 3.6 Epic EP-02: Payment and refund control

| Story | User story and acceptance criteria | Priority / estimate / dependency / owner |
|---|---|---|
| US-05 | As a customer, I want to see whether payment is pending, successful or failed so that I know what happened. AC: each attempt has a status and timestamp; pending is not shown as successful; retry guidance is clear. | Must / 5 points / US-02 / Integration engineer |
| US-06 | As a system, I want payment retries to use an idempotency key so that a timeout does not create a duplicate charge. AC: same key returns the existing attempt; duplicate payment record is not created; request and response references are stored. | Must / 8 points / US-05 / Integration + Backend |
| US-07 | As a customer, I want to request a return with a reason so that the retailer can review it. AC: return request links to order item; request status is visible; ineligible or duplicate requests are rejected. | Must / 5 points / US-02 / Frontend + Backend |
| US-08 | As a finance reviewer, I want to approve or reject a refund request so that money is not returned without authorization. AC: only authorized finance role can decide; approver, amount, reason and time are recorded; requester cannot approve own request. | Must / 8 points / US-07 / Backend + QA |
| US-09 | As finance, I want a refund record linked to the original payment so that reconciliation is possible. AC: refund amount cannot exceed eligible paid amount; refund status is stored; payment and refund references are traceable. | Must / 5 points / US-06, US-08 / Backend |

The payment sandbox is not expected until week four. Before then, the team builds and tests the payment interface against a deterministic fake adapter. The fake adapter is clearly labelled in the environment and cannot be mistaken for a live payment provider.

# Task 3 – Backlog Stories: Fulfilment and Support

## 3.7 Epic EP-03: Warehouse fulfilment and tracking

| Story | User story and acceptance criteria | Priority / estimate / dependency / owner |
|---|---|---|
| US-10 | As a picker, I want to see a pick list containing only reserved order items so that I do not pick stock for an unconfirmed order. AC: list shows order ID, item, quantity and location; cancelled/released orders are excluded; status updates are recorded. | Must / 8 points / US-03 / Frontend + Backend |
| US-11 | As a picker, I want to confirm picked quantities so that shortages are recorded rather than silently ignored. AC: picked quantity cannot exceed reserved quantity; short pick requires a reason; order is not dispatch-ready until required items are resolved. | Must / 5 points / US-10 / Frontend + QA |
| US-12 | As a warehouse user, I want to mark an order as dispatched so that tracking can begin. AC: dispatch requires completed picking; dispatch time and actor are stored; invalid status transition is rejected. | Must / 5 points / US-11 / Backend |
| US-13 | As a customer, I want to view order status changes so that I can understand fulfilment progress. AC: timeline is ordered by event time; internal-only notes are hidden; latest status is consistent with the order record. | Must / 5 points / US-12 / Frontend |

## 3.8 Epic EP-04: Support and visibility

| Story | User story and acceptance criteria | Priority / estimate / dependency / owner |
|---|---|---|
| US-14 | As a support agent, I want to search by order reference so that I can answer a customer enquiry. AC: exact order lookup returns customer-safe order, payment, dispatch and return status; access is role checked; sensitive payment details are masked. | Must / 5 points / US-02, US-05, US-13 / Frontend |
| US-15 | As an administrator, I want a basic report of order and refund states so that I can identify stuck work. AC: report counts orders by status and refunds by approval state; data is synthetic; report labels its time range. | Should / 3 points / core records / Database + Frontend |

A complete first release can be demonstrated by creating a synthetic order, reserving stock, recording payment status, moving the order through picking and dispatch, displaying the timeline, then submitting and approving a return/refund.

# Task 3 – Prioritization, Quality and Definitions

## 3.9 Two stakeholder trade-offs

First, warehouse staff may ask for barcode scanning before other fulfilment features. The scanner arrives only in week five, while reservation and pick-list correctness are needed earlier. The backlog therefore retains a manual pick confirmation story first and adds scanner input only if the device is available and the core flow is stable. This avoids making the entire picking process wait for hardware.

Second, finance needs strong refund approval and reconciliation, while support wants a complete console quickly. The support lookup is deliberately limited to customer-safe order and return information. Refund approval and audit persistence are prioritized before advanced support filters because an incorrect or untraceable refund has a greater financial and control impact.

## 3.10 Definition of Ready (DoR)

- The user and business reason are clear and the story is linked to an epic.
- Acceptance criteria are written in observable, testable language.
- Dependencies and external access requirements are identified.
- Required synthetic data, API contract and role permissions are understood.
- The story is small enough to complete within a sprint or is split.
- Privacy, accessibility and error-handling expectations have been considered.
- The product owner can answer outstanding questions before the story is selected.

## 3.11 Definition of Done (DoD)

- Code is reviewed by another team member and merged to the shared branch.
- Acceptance criteria pass in the agreed test environment.
- Relevant unit, integration and regression tests are run and results recorded.
- Authorization checks and sensitive-data masking are verified where applicable.
- Database changes preserve required order/payment/refund relationships.
- Keyboard navigation, readable labels and clear error messages are checked for user-facing screens.
- Logs and audit records do not expose card data or unnecessary personal information.
- Story is demonstrated and accepted by the product owner; documentation is updated.

# Task 4 – Create a Capacity-Based Release Plan and Delivery Workflow

## 4.1 Capacity assumptions

The team has seven members. For planning, each person is assumed to have 25 focused delivery hours per week after ceremonies, stakeholder meetings, support and normal interruptions. This gives 175 team hours per week or 350 hours per two-week sprint. The number is a planning ceiling, not a promise that all hours will be available.

The team has no established velocity. Therefore, story points are used only as relative estimates of effort, uncertainty and complexity. They are not converted into fixed hours. Sprint 1 is deliberately planned conservatively, and its completed work is used as an initial reference for Sprint 2.

| **Capacity item** | **Assumption** |
|---|---|
| Team size | 7 cross-functional members |
| Focused time per person | 25 hours/week |
| Nominal sprint capacity | 350 focused hours over two weeks |
| Planning reserve | Approximately 20% held for defects, clarification and integration |
| Initial planned story-point range | 18–22 points, subject to team discussion |
| Stakeholder time | 90 minutes per week for warehouse and finance clarification/review |

## 4.2 Estimation method

The team uses a modified Fibonacci scale (1, 2, 3, 5, 8, 13). During refinement, members compare a story with a small reference story and discuss complexity, uncertainty and testing effort. An 8-point story is not assumed to take twice as long as a 5-point story. Large stories are split when acceptance criteria cover unrelated outcomes.

## 4.3 Release sequencing

The order of work follows technical dependencies. Order and stock records are built first. Payment attempts and safe retries are added next, with the gateway sandbox introduced when available. Picking and dispatch depend on reliable reservations. Tracking and support lookup depend on order and fulfilment events. Return and refund control is built around the payment record and human approval.

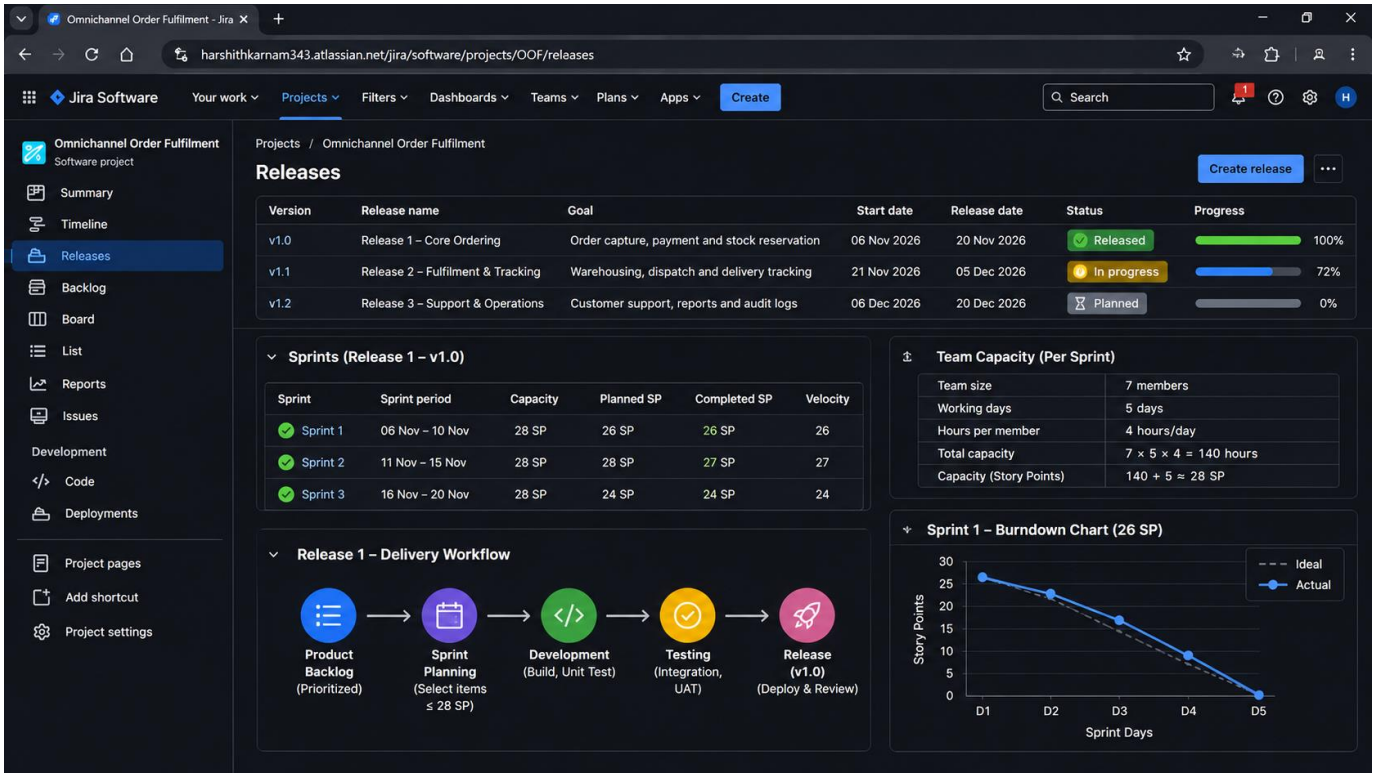
# Task 4 – Four-Sprint Release Plan

| Sprint | Goal | Main planned stories / outcome | Dependency notes |
|---|---|---|---|
| Sprint 1 (Weeks 1–2) | Create a reliable local order and stock foundation. | US-01, US-02, US-03 plus data model, basic tests and synthetic fixtures. | No live gateway or scanner required. Validate transaction and reservation rules. |
| Sprint 2 (Weeks 3–4) | Add payment state and a safe integration boundary. | US-05, US-06, payment adapter, timeout/retry tests and initial return request flow. | Sandbox access expected in week 4; continue fake adapter tests until access is confirmed. |
| Sprint 3 (Weeks 5–6) | Complete warehouse fulfilment and visible order progress. | US-10, US-11, US-12, US-13 and scanner trial if device is available. | Scanner arrives week 5; manual confirmation remains fallback. |
| Sprint 4 (Weeks 7–8) | Complete controlled refunds, support lookup and release validation. | US-08, US-09, US-14, selected US-15, regression and release rehearsal. | Requires payment and order history. Scope is frozen for final validation. |

## 4.4 Sprint goal checks

- Sprint 1 is successful when a synthetic order persists and stock cannot be over-reserved.
- Sprint 2 is successful when payment states and retry behaviour are testable with both fake and sandbox adapters where available.
- Sprint 3 is successful when reserved items can be picked, dispatched and displayed in a consistent timeline.
- Sprint 4 is successful when support can trace an order and finance can approve and audit a valid refund.

If a sprint has lower capacity than expected, the team removes Should-have work first. It does not remove authorization, idempotency, stock integrity or refund audit checks merely to keep a feature count high.



# Task 4 – Jira Workflow and Flow Policies

## 4.5 Proposed Jira development workflow

| Status | Entry condition | Exit condition |
|---|---|---|
| Backlog | Story is captured but not yet refined. | Product owner orders it and team refines it. |
| Selected for Sprint | Story meets DoR and is selected against capacity. | Owner starts work; dependencies are available. |
| In Progress | Implementation has started and an owner is assigned. | Code and developer checks are ready for review. |
| Code Review | Pull request is available and linked to the Jira issue. | Review comments resolved and changes merged. |
| Testing | Build is deployed to test environment. | Acceptance criteria and regression checks pass. |
| Blocked | A dependency, decision or environment prevents progress; blocked reason is recorded. | Blocker is removed and next action is assigned. |
| Done | All DoD conditions are satisfied and product owner accepts. | No further work unless a defect is reopened. |

## 4.6 WIP limits and transition rules

- Development: maximum two active stories at a time across the team.
- Testing: maximum two stories waiting for or undergoing QA.
- Code review: review should be taken before another large implementation task is started.
- Blocked: a story must include the blocker, responsible person and next review time.
- A story cannot move to Done only because coding is complete; test evidence and acceptance are required.
- Urgent work is not inserted silently. The product owner identifies what will be removed or moved out of the sprint.

## 4.7 Development workflow versus order lifecycle

The Jira workflow tracks the state of software work, such as In Progress, Code Review and Testing. The platform's operational order lifecycle is different: Created, Stock Reserved, Payment Pending/Confirmed, Picking, Dispatched, Delivered, Return Requested, Refund Approved/Rejected and Refunded. Keeping these two workflows separate prevents a Jira task status from being confused with a customer's order status.

## 4.8 Handling unfinished stories

At sprint end, an unfinished story returns to the backlog with its remaining work and blocker information visible. It is not automatically counted as completed or carried forward as a commitment. The product owner reorders it and the team checks whether its acceptance criteria need to be split.

# Task 5 – Simulate Delivery and Validate Your Design

## 5.1 Predicted bottleneck before simulation

The most likely first-sprint bottleneck is clarification and integration between checkout, the order database and stock reservation. The team has no established velocity, and a stock reservation error can affect every later stage. If the order data model or reservation rule is unclear, frontend work, test fixtures and later payment work will wait.

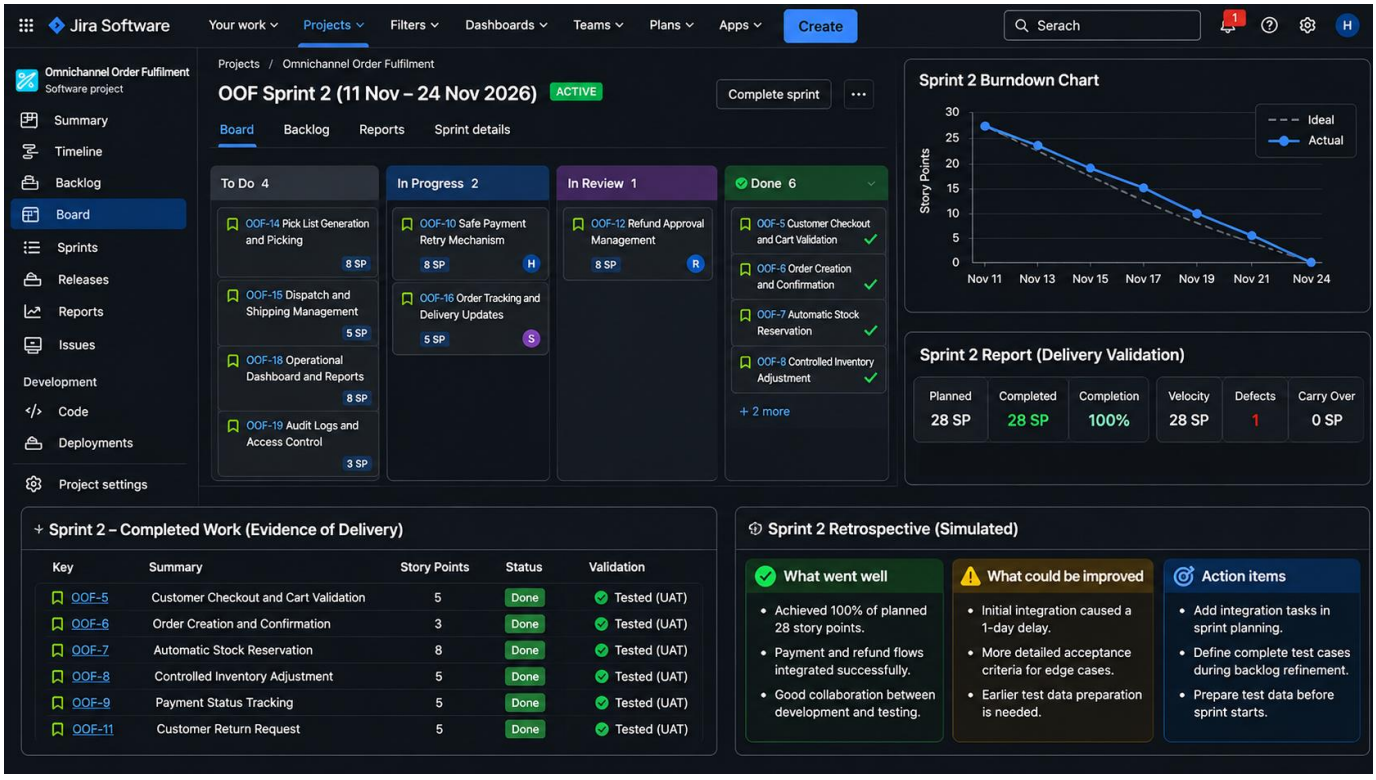
## 5.2 Control rule and evidence

The main control is to agree the order and inventory data contract before parallel implementation. The team uses a short design walkthrough, one accountable backend owner, a visible dependency link in Jira and a WIP limit of two active development stories. The rule is considered effective if blocked time stays low, the reservation story passes concurrent tests and no story is repeatedly reopened because of unclear data ownership.

## 5.3 Sprint 1 simulated event log

The following is a simulated sequence prepared to test the proposed process. It is not a record of an actual team sprint.

| Event | Simulated work event | Jira / process update |
|---|---|---|
| 1 | Sprint planning selects checkout, order persistence and reservation foundation. | Three stories moved to Selected for Sprint. |
| 2 | Backend owner starts the order data model. | US-02 moved to In Progress. |
| 3 | Frontend identifies that the order response format is not agreed. | US-01 marked Blocked; API clarification logged. |
| 4 | Team holds a short contract walkthrough and agrees response fields. | Blocker removed; API contract attached to story. |
| 5 | Reservation implementation begins using a database transaction. | US-03 moved to In Progress. |
| 6 | QA finds a test where two requests attempt to reserve the final unit. | Defect recorded and linked to US-03. |
| 7 | Backend adjusts the reservation transaction and adds a concurrency test. | Defect moved to Testing. |
| 8 | Checkout validation rejects a quantity greater than available stock. | US-01 acceptance test passes. |
| 9 | Order record and items persist with a unique reference. | US-02 passes integration test. |
| 10 | Sprint review demonstrates successful and failed reservation cases. | Product owner accepts the three simulated outcomes subject to regression. |



# Task 5 – Simulation Results and Improvement

## 5.4 Simulated delivery indicators

| Indicator | Simulated result | Interpretation |
|---|---|---|
| Planned versus completed | 3 stories planned; 3 accepted after correction. | The small initial scope was achievable in this simulation. |
| Blocked time | US-01 blocked for 0.5 working day while API fields were clarified. | Contract clarification created a short delay but made the dependency visible. |
| Cycle time | US-02: 3 working days; US-01: 4 working days; US-03: 5 working days. | Reservation had the longest cycle because of concurrency testing. |
| WIP | Peak 3 stories active briefly; target limit was 2. | The limit was not consistently followed during parallel start-up. |
| Reopened stories | 1 reservation defect returned from testing to development; 0 stories reopened after final acceptance. | Early defect discovery was useful, but concurrency cases should be in the initial test plan. |

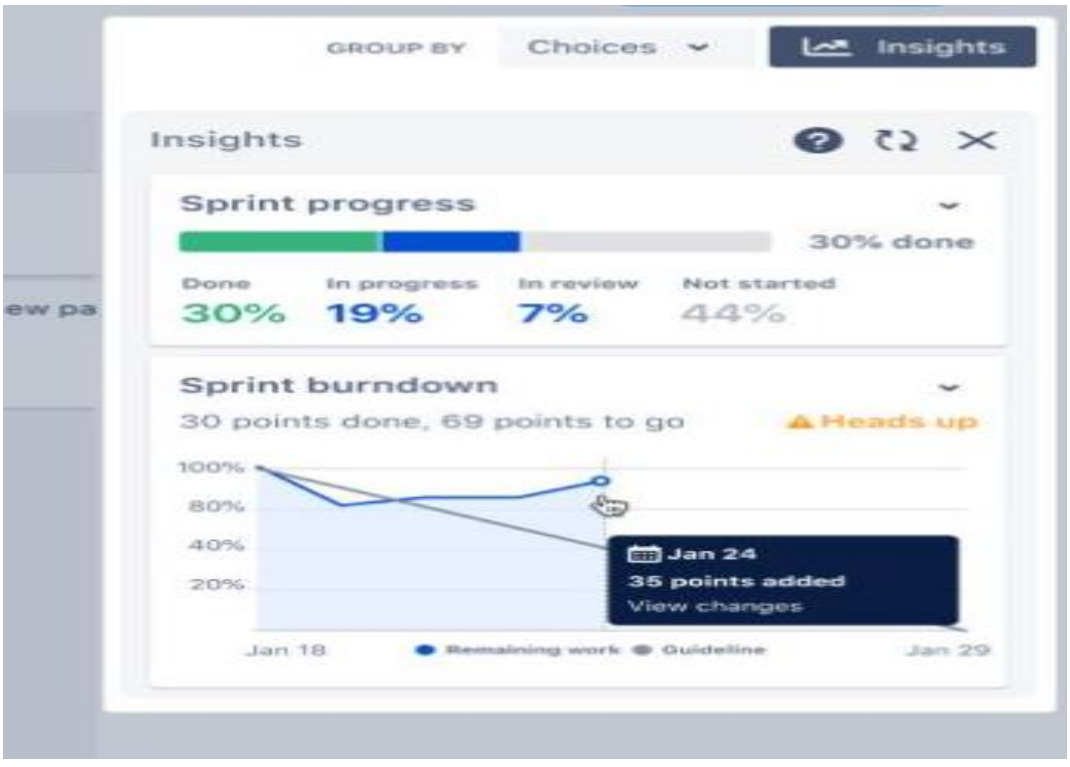
## 5.5 What the results suggest

The simulated result suggests that the chosen scope can be completed when the team starts with a narrow foundation. However, the event log exposes two weaknesses. First, the API contract was clarified only after frontend work had started. Second, the WIP limit was exceeded briefly because team members began separate tasks before finishing the shared dependency. These are process issues rather than evidence that the whole plan is unworkable.

## 5.6 Improvement for Sprint 2

- Add a short API and data-contract review to refinement for every cross-layer story.
- Require a named dependency owner before a story enters Selected for Sprint.
- Keep the development WIP limit at two and ask the team to swarm on a blocked item before starting another.
- Add concurrent reservation and duplicate-request cases to the standard regression suite.
- Review the blocked-time and cycle-time trend at the end of Sprint 2 rather than judging only story points.

The simulation is useful because it changes the plan based on observed process behaviour. The team should not increase the next sprint commitment solely because all three simulated stories were completed.



# Task 6 – Redesign Delivery Under Changed Conditions

## 6.1 Changed requirements

At the end of the first simulated sprint, two additional regional warehouses are added, a second seven-member team joins, the original payments engineer becomes unavailable, and region-specific tax, shipping and refund-approval rules become necessary. The festival-sale deadline does not move. These changes affect both the technical design and the way work is coordinated.

## 6.2 Impact diagnosis

| Change | Affected original assumption | Impact and risk |
|---|---|---|
| Two regional warehouses | One representative warehouse and shared stock rules were assumed. | Stock, shipping and fulfilment behaviour now vary by region; global assumptions may create incorrect reservations. |
| Second seven-member team | One seven-member team owned the backlog and integration. | More capacity is available, but parallel work can increase coordination cost and merge conflicts. |
| Payments engineer unavailable | Payment adapter had one specialist owner. | Payment work becomes a critical skill gap; knowledge transfer and a backup owner are required. |
| Regional tax/shipping/refund rules | One general pricing and refund rule was assumed. | Order totals and refund eligibility must use a region-specific rule set and be auditable. |
| Deadline unchanged | Four sprints remain within eight weeks. | There is no extra time for broad redesign; scope must be narrowed and integration sequenced. |

## 6.3 Revised risk assessment

The highest risk is now the combination of regional rule complexity and the loss of the payments specialist. The second team does not automatically double useful delivery capacity because both teams need shared decisions, integration environments and a stable data model. A scaled approach must reduce dependencies rather than create two teams building overlapping parts of the same transaction.

# Task 6 – Redesign Objectives and Team Coordination

## 6.4 Measurable redesign objectives

| Objective | Target by release | Validation |
|---|---|---|
| Regional correctness | All scripted orders use the correct region's tax and shipping rule. | Test matrix with at least two regions and boundary values. |
| Refund control | Every refund follows the correct region's approval rule and retains approver evidence. | Role and audit tests for each configured rule. |
| Payment continuity | A second engineer can build, test and explain the payment adapter without the unavailable engineer. | Pairing notes, code walkthrough and successful retry tests. |
| Cross-team integration | One integrated build is produced at least twice per sprint. | Shared integration pipeline and review record. |
| Deadline protection | Release contains a complete order-to-delivery-to-refund journey for supported regions, with nonessential features deferred. | Sprint goal and end-to-end release rehearsal. |

## 6.5 Revised team structure

| Team | Primary ownership | Shared responsibilities |
|---|---|---|
| Team A – Core transaction | Order, inventory reservation, payment adapter, common data model. | API contracts, security rules, integration testing. |
| Team B – Regional fulfilment and experience | Regional configuration, warehouse workflow, tracking and support screens. | End-to-end tests, documentation and release readiness. |
| Product owner / business representatives | One ordered product backlog and region rule decisions. | Prioritization and acceptance across both teams. |
| Delivery lead / Scrum facilitators | Coordinate sprint goals, dependencies and impediments. | Cross-team planning and integrated review. |

The teams remain cross-functional rather than splitting strictly into frontend and backend groups. Each team should be able to complete a vertical slice, while shared specialists review architecture and security. The payments work is paired with a backend engineer from Team A and a reviewer from Team B to reduce single-person dependency.

## 6.6 Scaled Agile practices adopted

- One shared product backlog with one product owner, not separate competing priority lists.
- Joint planning at the start of each two-week sprint to identify cross-team dependencies.
- A short weekly cross-team sync focused only on interfaces, blockers and integration risks.
- A shared Definition of Done and common release branch/integration pipeline.
- A joint sprint review demonstrating one integrated order journey across the configured regions.
- Retrospectives within each team plus a short joint review of coordination issues.

# Task 6 – Revised Backlog and Sequencing

## 6.7 Revised backlog decisions

| Priority | Revised item | Reason / owner |
|---|---|---|
| Must | Region identifier stored with each order and shipment. | Required to apply the correct rules; Team A. |
| Must | Configurable tax calculation by region with testable rate/rule table. | Prevents incorrect customer totals; Team A + finance reviewer. |
| Must | Shipping method and charge selected using region configuration. | Needed for regional fulfilment; Team B. |
| Must | Refund approval policy selected by region and role. | Protects finance control; Team A + finance reviewer. |
| Must | Payment adapter ownership transfer, documentation and retry tests. | Addresses unavailable specialist; Team A. |
| Must | Regional warehouse stock reservation and pick-list filtering. | Prevents picking from the wrong warehouse; Team B. |
| Should | Scanner integration for both warehouses. | Only after device and core manual workflow are stable; Team B. |
| Defer | Advanced analytics, forecasting and personalization. | Not required to protect the core journey by the fixed date. |

## 6.8 Revised sequencing

The first sprint has already established the local order and stock foundation in the simulation. In Sprint 2, the teams add the region field and rule configuration before expanding the payment integration. This prevents tax and shipping rules from being bolted onto completed order records later. The payment adapter is documented and paired early in the sprint, while the second team builds regional fulfilment against the agreed contract.

In Sprint 3, both teams integrate regional stock reservation, picking and tracking. Scanner work is optional and cannot delay manual picking. In Sprint 4, refund approval is validated against regional rules, support lookup is checked for role-based visibility and the complete journey is rehearsed. Any region that cannot pass the agreed end-to-end tests is excluded from the release rather than released with an unsafe partial workflow.

## 6.9 Lean coordination

- Use one shared board and explicit team labels; avoid duplicate tickets for the same outcome.
- Limit each team to two active development stories and two testing items.
- Track cross-team dependencies with linked Jira issues and a named owner.
- Integrate small changes daily or at least several times per week to avoid a late merge event.
- Use a visible blocked reason and escalate a blocker that lasts more than one working day.

# Task 6 – Validate the Redesign

## 6.10 Additional simulated events

The following five-plus events are simulated to compare the redesigned approach with the original plan. They are not actual delivery records.

| Event | Simulated event after change | Redesign response |
|---|---|---|
| 1 | Regional finance clarifies that one region requires a different refund approval role. | Add region-policy story and approval matrix before implementation. |
| 2 | The original payments engineer becomes unavailable. | Pair a second backend engineer with the integration owner; conduct code walkthrough. |
| 3 | Team B begins a warehouse screen but discovers the order API lacks a region field. | Shared contract review; add region field to the core model before further UI work. |
| 4 | Tax calculation test fails at a regional boundary value. | Add boundary tests and make tax rule configuration data-driven. |
| 5 | Two teams attempt changes to the same order status service. | Assign service ownership and require API review before merging. |
| 6 | Scanner device arrives, but manual picking already passes the core tests. | Run a limited scanner trial without making it a release dependency. |
| 7 | Joint release rehearsal finds a refund approval permission gap. | Block release candidate; fix authorization and repeat refund audit tests. |

## 6.11 Comparative indicators (simulated)

| Indicator | Original single-team simulation | Redesigned two-team simulation |
|---|---|---|
| Planned versus completed | 3 of 3 small foundation stories accepted. | 5 of 6 revised priority stories accepted; one optional scanner story deferred. |
| Blocked time | 0.5 working day for API clarification. | 1.0 working day across shared API and payment knowledge transfer. |
| WIP | Peak 3 active stories against limit of 2. | Peak 4 across two teams; each team stayed within its local limit, but shared integration queue briefly exceeded target. |
| Reopened / failed checks | 1 reservation defect corrected before acceptance. | 2 issues found in integration (tax boundary and refund permission); both corrected before release rehearsal. |
| Integration cadence | One team integrated within its sprint. | Joint build at least twice per sprint; one shared-service conflict required ownership clarification. |



# Task 6 – Defence of the Redesign and Task 7 – Individual Technical Reflection

## 6.12 Defence of the revised strategy

The redesign is more feasible than simply adding all new requirements to the original backlog because it changes the data model and rule ownership early, gives the teams clear boundaries and keeps one shared priority order. The second team provides additional delivery capacity, but the plan does not assume that capacity doubles. Coordination, onboarding and integration consume time, and the payments specialist's absence creates a knowledge risk.

The main trade-off is that advanced analytics and full scanner automation are deferred. This is acceptable because the fixed deadline is tied to a complete and controlled fulfilment journey. The first release should support only regions that pass the tax, shipping, stock and refund test matrix. A smaller correct release is preferable to a broader release that can calculate the wrong amount or approve a refund under the wrong rule.

## 7. Individual Technical Reflection

The most significant decision in this assignment was prioritizing a complete order journey instead of attempting all nine proposed modules. This decision connected the backlog to the retailer's actual problems. Checkout without stock reservation would still allow failed orders, and a tracking screen without reliable dispatch events would only display uncertain information. The first release therefore gives priority to the records and controls that make later screens trustworthy.

The simulated Sprint 1 event log influenced the strategy by showing that the API contract was clarified too late and that the WIP limit was briefly exceeded. I added contract review to refinement, assigned a dependency owner and kept the team focused on finishing blocked work. The changed-condition simulation also showed why shared service ownership and early integration are necessary when two teams work on the same platform.

The final design remains vulnerable to late changes in regional policy, incomplete gateway sandbox behaviour and inaccurate capacity assumptions. It also depends on the product owner receiving timely decisions from finance and warehouse representatives. Before using this plan with a real team, I would validate the regional rules with the actual business, inspect the gateway's retry and refund behaviour, run a short technical spike and measure the team's real availability.

An ethical concern addressed in the design is the handling of customer and payment information. The prototype uses synthetic records and does not store card data. Support users see only the information needed to answer an enquiry, while refund approval is restricted to an authorized finance role. Accessibility is considered through readable labels, keyboard operation and clear status messages. Workload transparency is supported by WIP limits and realistic capacity assumptions rather than pressuring individuals to maximize story points.

## References and acknowledgement


- SIMATS Engineering, CSA10 – Software Engineering, Assignment 1 brief: Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform (provided course document).
- Atlassian Jira product documentation – used as a reference for the proposed issue, backlog, board and workflow concepts.
- Scrum Guide (2020) – used for the general Scrum roles, events and product backlog concepts.
- All stakeholder responses, capacity values, targets, event logs and simulation indicators in this report are explicitly constructed planning assumptions, not external measurements.


End of assignment report.